

AI Disease Risk Prediction Project

Beginner-friendly step-by-step explanation of the current project.

1. Big Picture

This project is meant to predict disease risk using machine learning. At the moment, it focuses on diabetes prediction using the dataset **data/diabetes.csv**. The main work is inside the notebook **notebooks/eda.ipynb**.

2. Current Project Files

Path	Purpose
README.md	Short description of the project idea.
data/diabetes.csv	Diabetes dataset used for training and testing.
notebooks/eda.ipynb	Notebook containing data analysis and model training.
reports/	Folder for generated reports such as this PDF.
models/	Placeholder folder for saved trained models.
dashboard/	Placeholder folder for a future dashboard or app.
requirement.txt	Currently empty; should later list Python dependencies.

3. The Dataset

The dataset has 768 patient records. Each row represents one person. The columns describe health measurements and the final result.

```
Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI,  
DiabetesPedigreeFunction, Age, Outcome
```

The most important column is **Outcome**: 0 means no diabetes, and 1 means diabetes. In this dataset, there are 500 records with Outcome 0 and 268 records with Outcome 1.

4. Loading the Data

The notebook uses Pandas to load the CSV file into a DataFrame. A DataFrame is like an Excel table inside Python.

```
import pandas as pd  
df = pd.read_csv(r'D:\PROJECTS\AI_Disease_Risk_Prediction\data\diabetes.csv')  
df.head()
```

5. Exploring the Data

The notebook creates charts to understand the data before training a model.

- **Count plot:** shows how many people have diabetes and how many do not.
- **Correlation heatmap:** shows which columns are related to each other and to the final result.

6. Separating Inputs and Answer

Machine learning needs inputs and an answer. In this project, X contains the health measurements, and y contains the correct diabetes result.

```
x = df.drop('Outcome', axis=1)
y = df['Outcome']
```

7. Splitting Training and Testing Data

The dataset is split into two parts: 80% for training and 20% for testing. Training data teaches the model. Testing data checks whether the model learned properly.

8. Training Logistic Regression

Logistic Regression is a beginner-friendly classification model. It is used when the answer is a category, such as diabetes or no diabetes.

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
```

9. Making Predictions

After training, the model predicts diabetes outcomes for the test data.

```
y_pred = model.predict(X_test)
```

10. Evaluating the Model

The notebook evaluates the model using accuracy, confusion matrix, and classification report.

- **Accuracy:** percentage of predictions that were correct.
- **Confusion matrix:** shows correct and incorrect predictions for both classes.
- **Classification report:** shows precision, recall, f1-score, and support.

11. Training Random Forest

The notebook also trains a Random Forest model. A Random Forest is made from many decision trees, and it often performs better than a single simple model.

```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
```

12. Comparing Models

The project tries to compare Logistic Regression and Random Forest. There is one small notebook mistake: the comparison cell prints Random Forest accuracy for both models.

```
Correct version:
print('Logistic Regression Accuracy:', accuracy)
print('Random Forest Accuracy:', rf_accuracy)
```

13. Recommended Notebook Order

- 1 Import libraries.

- 2 Load the dataset.
- 3 Explore the data with charts.
- 4 Create X and y.
- 5 Split the data into training and testing sets.
- 6 Train Logistic Regression.
- 7 Predict and evaluate Logistic Regression.
- 8 Train Random Forest.
- 9 Predict and evaluate Random Forest.
- 10 Compare both models.

14. Current Project Status

Right now, this is mainly a machine learning notebook project for diabetes prediction. It is not yet a complete application. A complete version would usually add a cleaned notebook, a saved trained model, a Streamlit app, generated reports, and a filled requirement.txt file.